

PlayTools 技术报告

注入式 iOS-on-Mac 运行时与 MaaTools 截图服务

代码调查报告

2026 年 9 月 1 日

摘要

PlayTools 是 PlayCover 用于增强 iOS 应用在 macOS 上运行能力的外部 framework/dylib。本报告以 PlayCover 的 **MaaTools** 分支和其锁定的 PlayTools revision 为对象，重点说明 PlayTools 的注入方式、目标进程内的初始化链、**AKInterface** 窗口桥接、MaaTools TCP 协议及其截图像素输出。调查结论是：PlayCover 主程序负责设置、注入、签名和启动；真正的窗口采集及 TCP 服务运行在被注入的目标应用进程内，不由 PlayCover 主进程的菜单或截图 API 完成。服务返回内存中的原始像素，不保存 PNG/JPEG 文件。

目录

1	调查范围与证据边界	2
1.1	对象与版本	2
1.2	证据标签	2
2	总体架构与进程边界	2
3	PlayCover 侧的安装、注入与配置	3
3.1	安装决策	3
3.2	目标应用启动前检查	4
3.3	MaaTools 配置	4
4	目标进程内的运行时入口	4
5	窗口图像采集流程	5
5.1	窗口识别与采集后端	5
5.2	标题栏裁剪与两种像素输出	5
6	MaaTools TCP 协议	5
6.1	握手与请求帧	5

目录	2
6.2 命令与响应	6
7 配置生命周期与异常边界	6
7.1 两侧设置模型	6
7.2 权限边界	6
7.3 调试捕获变量不是 MaaTools	7
8 安全性与可靠性评估	7
9 分支差异与构建限制	7
10 结论	8
A 源码证据索引	8
B 验证状态与后续测试矩阵	9

1 调查范围与证据边界

1.1 对象与版本

本报告调查的宿主仓库是 PlayCover 的 MaaTools 分支,当前提交为 63cf605,对应版本标记 v3.1.0.maa.7。该分支的 Cartfile:1 指向 hguandl/PlayTools 的 MaaTools 分支; Cartfile.resolved:1 将依赖固定到 revision 7aa1977。完整提交标识如下:

```
PlayCover HEAD: 63cf605d209de834a333b09c85ca5bea261dc9eb
PlayTools revision: 7aa19776dbb72b0ae7f0989568f69c7c9c2d6543
```

PlayCover 工程的 macOS deployment target 为 12.0。当前检出目录没有 Carthage/Build, 因此生成的 PlayTools.xcframework 和 AKInterface.bundle 不属于本地版本库的源码文件。外部 PlayTools 的静态描述仅以上述固定 revision 的源码为依据;下文会明确区分本地证据与外部证据,避免把未生成的二进制行为误写成当前本地可运行的结果。

1.2 证据标签

- [L]: PlayCover 仓库中的本地静态代码或配置。
- [X]: 固定 revision 的外部 PlayTools 源码。
- [R]: 需要运行已构建 framework、目标应用或 TCP 客户端才能验证的事实。
- [I]: 根据代码结构作出的分析判断,不等同于已执行的运行时测试。

2 总体架构与进程边界

PlayTools 不是一个由 PlayCover 主进程通过菜单调用的独立截图工具,而是随目标应用启动、在目标应用地址空间中运行的注入式组件。其主要边界如下:

组件或边界	责任
PlayCover 主程序	提供 SwiftUI 设置界面,保存 plist, 安装和签名 PlayTools, 修改目标 Mach-O, 并启动目标应用。
目标应用进程内的 PlayTools.dylib	通过 Mach-O 加载命令被注入; 包含 PlayLoader、PlayCover.launch 和 MaaTools 初始化逻辑。
目标应用内的 AKInterface.bundle	由插件加载器加载, 桥接 macOS 窗口属性、窗口图像、窗口内容区域和终止操作。
外部 MAA 客户端	通过 TCP 连接目标进程内的 MaaTools, 发送命令并自行解码原始像素或执行支持的输入控制。

运行关系可以简化为：

```

PlayCover host process
+-- AppSettings / Installer / signings and injection
`-- launch target application
    |
    v
Target iOS-on-Mac process
+-- PlayTools.dylib
|   `-- PlayLoader constructor
|       `-- PlayCover.launch
|           `-- MaaTools.initialize
`-- AKInterface.bundle
    `-- windowImage / windowContentRect / terminateApplication
        |
        v
MAA client ----- TCP ----- MaaTools

commands: SCRN / BGR\1 / SIZE / VERN / BNDL / RECT / TOUCH / TERM

```

该边界解释了三个容易混淆的对象：iTunes 元数据中的 `screenshotUrls` 只是数据模型字段，不是截图服务；`METAL_*` 环境变量清理属于调试环境管理，也不是 MaaTools 截图生成；PlayCover 主进程本身没有截图文件管理 API。

3 PlayCover 侧的安装、注入与配置

3.1 安装决策

安装 IPA 时，PlayCover/PlayCover/AppInstaller/Installer.swift:11-35 弹窗询问是否安装 PlayTools，并将用户选择保存为安装偏好。PlayCover/PlayCover/AppInstaller/Installer.swift:94-98 在导出场景调用 `PlayTools.injectInIPA`；在普通安装场景中，只有用户选择安装时才调用 `PlayTools.installInIPA`。用户选择不注入，目标应用就不会具备后文所述的运行时服务。

PlayTools 的宿主侧安装方法会把内置 framework 复制到用户的 framework 目录：PlayCover/PlayCover/Utils/PlayTools.swift:45-70。针对 IPA 的安装流程包括：

1. 使用注入库向目标 Mach-O 添加 PlayTools 的 dylib 加载命令；
2. 复制本地化资源和 `AKInterface.bundle`；
3. 清除可能导致 codesign 失败的扩展属性；
4. 修正插件可执行权限并对插件签名；
5. 对处理后的目标应用或其 Mach-O 重新签名。

PlayCover/PlayCover/Utils/PlayTools.swift:73-125 实现了 framework 路径下注入 dylib、复制插件和签名的主要步骤。工程构建阶段还从 Carthage/Build/PlayTools.xcframework 复制 framework，并将 `AKInterface.bundle` 复制到 framework 的插件目录：

PlayCover/PlayCover.xcodeproj/project.pbxproj:587, 597–599。

3.2 目标应用启动前检查

PlayCover/PlayCover/Model/PlayApp.swift:63–115 的 `launch()` 在启动前同步设置、检查 entitlements 和 Info.plist 签名、解锁必要的 keychain 状态，并检查 PlayTools 是否已安装。若检测到目标应用已有 PlayTools，PlayCover/PlayCover/Model/PlayApp.swift:92–95 会补齐插件；如果 `isInstalled()` 返回失败，则记录错误并不会进入 `runAppExec()`。因此截图服务的第一道前置条件是 PlayTools 已正确安装或注入。

3.3 MaaTools 配置

宿主侧的 `AppSettingsData` 定义了：

```
maaTools      = false
maaToolsPort = 1717
```

对应代码位于 PlayCover/PlayCover/Model/AppSettings.swift:9–31, 104–169。配置以 XML property list 保存到：

```
~/Library/Containers/io.playcover.PlayCover/App Settings/<bundle-id>.plist
```

设置页的 Bypasses tab（包括 MaaTools）在没有 PlayTools 时禁用，相关逻辑位于 PlayCover/PlayCover/Views/AppSettingsView.swift:78–107；MaaTools 开关和端口控件位于 PlayCover/PlayCover/Views/AppSettingsView.swift:539–586，端口 UI 范围为 1024–65535。

4 目标进程内的运行时入口

外部 PlayTools 的 PlayLoader.m:332--334 使用 constructor：

```
static void __attribute__((constructor)) initialize(void) {
    [PlayCover launch];
}
```

这意味着 dylib 一旦被目标进程加载，初始化会自动发生，不需要 PlayCover 主进程再次发送“开始截图”命令。固定 revision 中的 PlayCover.swift:13--23 依次初始化 `AKInterface`、`PlayScreen`、`PlayInput` 和其他运行时组件，再在主线程异步调用 `MaaTools.shared.initialize`。

MaaTools 初始化首先检查 `PlaySettings.shared.maaTools`。如果关闭则直接返回；如果开启，则每秒检查一次 key window、native scale、native bounds 和窗口标题，直到宽高非零且标题可用，然后创建 TCP listener。该等待循环没有超时，目标窗口永久不满足条件时，初始化任务也不会自然结束。

listener 就绪后，服务端将窗口标题修改为原窗口标题加上 `[localhost:<port>]`。这个标题后缀是可观察的启动副作用，但不能据此证明 socket 只绑定 loopback 地址。

5 窗口图像采集流程

5.1 窗口识别与采集后端

`AKInterface` 首先取得目标应用第一扇窗口的 `CGWindowID`。macOS 14.4 及以上使用 `ScreenCaptureKit`：

- 以当前进程的可共享内容查找相同 window ID；
- 使用独立窗口过滤器和 `SCScreenshotManager.captureImage`；
- 配置 32-bit BGRA、sRGB、关闭光标、忽略阴影和全局裁剪，并选择最佳采集分辨率。

macOS 14.4 以下使用 `CGWindowListCreateImage`，并设置最佳分辨率、忽略窗口边框和不透明选项。需要注意，现代路径的 `ScreenCaptureKit` 异常只记录错误并返回空结果，不会在同一次调用中自动回退到旧 API。`PlayCover` 的 deployment target 是 12.0，因此两条路径都会被保留。

5.2 标题栏裁剪与两种像素输出

截图逻辑每次从 `AKInterface.windowImage` 获取 `CGImage`。它使用窗口宽高比例计算标题栏高度，并将顶部标题栏裁掉，使输出尺寸对应目标应用的内容区域。

SCRN 分配 $4 \times width \times height$ 字节的 buffer，使用 sRGB `CGContext` 将图像绘制到 4 bytes-per-pixel 的原始缓冲区，再返回该缓冲区。代码没有进行 PNG/JPEG 编码。

BGR\1 通过 `Accelerate/vImage` 将图像的 `RGBA8888` 数据转换为去 alpha 的 `RGB888` 数据。虽然命令名含有 BGR，当前实现使用的是 `vImageConvert_RGBA8888toRGB888`，报告应按实现记录为 `RGB888` 原始字节，而不是根据命令名推断通道顺序。

如果无法取得图像，普通截图返回长度为零的数据；BGR 响应则返回零尺寸和空数据。失败主要通过 `OSLog` 记录，`PlayTools` 没有本地文件落盘路径、命名规则或图像扩展名。

6 MaaTools TCP 协议

6.1 握手与请求帧

固定 revision 的协议版本常量为 3。新连接先读取 4 字节握手：客户端必须发送 `MAA\0`，服务端回复 ASCII `OKAY`。握手之后，读取循环先读取 2 字节大端序长度，再读取相应长度的 payload。payload 的前 4 字节是命令 magic。

```
connect:
    client  -> MAA\0
    server  -> OKAY

request frame:
    u16be payload_length || payload
    payload[0..3] = command magic
```

6.2 命令与响应

命令	当前实现的行为
SCRN	返回 <code>u32be(data_length)</code> ，后接每像素 4 字节的 sRGB 原始数据。
BGR\1	返回 <code>u32be(width)</code> 、 <code>u32be(height)</code> 、 <code>u32be(data_length)</code> ，后接 RGB888 原始数据。
SIZE	返回两个 16-bit 大端序整数：窗口内容宽度和高度。
VERN	返回 32-bit 大端序协议版本，目前为 3。
BNDL	返回长度和目标应用 bundle identifier 的 UTF-8 字节。
RECT	返回窗口 frame 与 content rect 的坐标和尺寸字段。
TOUCH	从 payload 中读取阶段与坐标，派发 down、move 或 up 触控事件。
TERM	调用 <code>AKInterface.terminateApplication()</code> 终止目标应用。

截图响应中的长度字段和像素数据是协议数据，不是文件格式。客户端需要知道宽度、高度、每像素字节数和通道排列，完成内存解码后才能显示或自行保存成图片。服务端本身不提供文件名、扩展名、PNG/JPEG 编码或文件路径。

7 配置生命周期与异常边界

7.1 两侧设置模型

外部 PlayTools 的 `PlaySettings.swift:13--23` 根据当前用户名拼出与宿主相同的 plist 路径，并一次性解码 `maaTools` 与 `maaToolsPort`。文件读取或解码失败时回退到默认设置，其中 `maaTools=false`、端口为 1717，并打印失败信息。由于 MaaTools 是目标进程 constructor 启动后初始化，设置改变通常要在目标应用下一次启动时才会生效。

服务端通过 `UInt16(maaToolsPort & 0xffff)` 生成端口；正常 UI 范围可避免大部分非法输入，但端口冲突和 listener 创建失败仍需单独处理。代码未指定 host，也没有认证或 TLS。`NWListener` 的实际绑定范围应在具备构建产物环境中通过 socket 检查确认，不能仅凭窗口标题中的 localhost 判断。

7.2 权限边界

`PlayCover/PlayCover/Utils/Entitlements.swift:45-65` 为目标应用生成的基础 entitlements 包含 `com.apple.security.network.server`；这是网络服务能力，与屏幕采集授权不是同一个机制。PlayCover 自身的 `PlayCover.entitlements` 和 `PlayCoverRelease.entitlements` 为空或接近为空，不代表目标应用已经获得屏幕录制许可。

`PlayCover/PlayCover/Info.plist:78-81` 只有 `NSSystemAdministration` `UsageDescription`，未发现显式 `Screen Recording UsageDescription`、屏幕采集 entitlement 或 TCC 授权请求 API。因此 `ScreenCaptureKit` 最终能否成功，仍取决于 macOS 的 TCC 状态和用户实际授予的屏幕录制权限。当前代码对拒绝或采集失败主要记录错误并返回空数据。

7.3 调试捕获变量不是 MaaTools

PlayCover/PlayCover/Model/PlayApp.swift:123–150, 156–162 在启动目标应用前清理 DYLD_*、METAL_CAPTURE_*、METAL_FRAME_CAPTURE_ENABLED 和 MTLCaptureEnabled 等变量。这是为了避免调试 wrapper、验证层或 Metal Frame Capture 配置从父进程继承；它不生成 MaaTools 截图，且可能使另一个独立的 Metal 调试流程失效。

8 安全性与可靠性评估

以下项目是基于源码的分析建议，不应理解为当前实现已经具备这些防护：

1. **限制网络暴露。** listener 创建时没有显式 host、认证或加密。应明确绑定 loopback，或引入认证令牌和 TLS；同时对“未指定 host”的实际行为做运行时确认。
2. **区分截图与控制权限。** 同一连接除了读取截图，还支持 TOUCH 和 TERM。若服务可被非预期客户端访问，可能造成输入注入或远程终止，建议将控制命令单独授权。
3. **增加资源边界。** 请求长度字段是 16-bit，但代码未体现连接级超时、来源校验、速率限制或明确的应用层超大请求策略；应在客户端断开、半包、端口冲突和重复连接场景加入测试。
4. **处理窗口生命周期。** 无限窗口轮询可能使初始化任务长期存在；窗口关闭、重建或应用异常退出时，listener 的取消和资源释放也应有明确策略。
5. **改善权限反馈。** ScreenCaptureKit 失败目前主要表现为日志或空数据。宿主可以在设置或首次连接时给出 TCC 权限状态和可操作的错误信息。
6. **固定并审计依赖。** 发布构建应继续锁定 PlayTools revision，同时保留协议版本、像素格式、权限行为和签名流程的回归测试。

9 分支差异与构建限制

当前 MaaTools 分支不是 PlayCover 主线的普遍能力：

- 当前 Cartfile:1 指向 hguandl/PlayTools 的 MaaTools 分支，并在 Cartfile.resolved:1 固定 revision。
- 对比的 origin/develop 仍使用 PlayTools 的另一版本，不包含当前 maaTools 和 maaToolsPort 宿主设置链。
- 提交 74f55d3 引入了 MaaTools 设置字段与 UI；因此切换到没有这些改动的分支后，不能假定截图 TCP 服务仍存在。
- 构建脚本依赖 Carthage 生成的 PlayTools.xcframework，并对 framework、插件和目标应用执行多步复制、转换及签名。当前工作树没有 Carthage/Build，无法仅凭宿主仓库完成真实截图服务的端到端运行验证。
- 仓库内未发现 XCTest、截图测试或其他专门测试目录；协议解码、TCC 拒绝、端口冲突和关闭生命周期目前缺少本地自动化覆盖。

10 结论

PlayTools 的核心价值不在于提供一个 PlayCover 菜单截图功能，而在于把运行时能力注入目标 iOS-on-Mac 应用。MaaTools 是其中一个可选服务：目标 dylib 被加载后自动初始化，读取宿主生成的 plist，等待 key window 就绪，调用 AKInterface 获取窗口图像，并通过 TCP 返回原始像素。PlayCover 负责让这条链路成立——安装、注入、签名、配置、目标应用启动——但截图采集与协议处理本身属于外部 PlayTools。

当前实现适合由受信任的本机客户端使用，但应谨慎对待网络边界和控制命令：协议没有内建认证，服务还支持触控和终止操作，ScreenCaptureKit 又依赖外部 TCC 授权。若要将 MaaTools 用作稳定的自动化截图接口，优先级应是确认 socket 绑定范围、增加连接认证与超时、为协议和生命周期补测试，并提供清晰的屏幕录制权限反馈。

A 源码证据索引

标签	文件或固定 revision	支持的事实
[L]	PlayCover/PlayCover/Model/PlayApp.swift:63–115	目标应用启动、PlayTools 检查、插件安装和执行入口。
[L]	PlayCover/PlayCover/AppInstaller/Installer.swift:11–35, 94–98	安装时是否注入 PlayTools 的用户选择与分支。
[L]	PlayCover/PlayCover/Utils/PlayTools.swift:45–70, 73–125	framework 安装、dylib 注入、插件复制和签名。
[L]	PlayCover/PlayCover/Model/AppSettings.swift:9–31, 104–169	MaaTools 默认值、端口和 plist 保存位置。
[L]	PlayCover/PlayCover/Views/AppSettingsView.swift:78–107, 539–586	无 PlayTools 时禁用设置页，以及 MaaTools 控件。
[L]	PlayCover/PlayCover/Utils/Entitlements.swift:45–65	目标应用生成的网络服务 entitlement。
[L]	PlayCover/PlayCover/Info.plist:78–81	当前 Info.plist 的权限描述范围。
[L]	PlayCover/Cartfile.resolved:1	PlayTools 固定 revision。
[L]	PlayCover/PlayCover.xcodeproj/project.pbxproj:587, 597–599, 801, 839	Carthage framework 复制脚本和 macOS deployment target。
[X]	hguandl/PlayTools@7aa1977:PlayLoader.m, 332–334 行	dylib constructor 自动调用 PlayCover.launch。
[X]	同一 revision:PlayCover.swift, 13–23 行	运行时初始化顺序及异步启动 MaaTools。

标签	文件或固定 revision	支持的事实
[X]	同一 revision: MaaTools.swift, 12-216 行	版本、握手、请求帧、listener 和 SCR_N。
[X]	同一 revision: MaaTools.swift, 284-328 行	RGB888 转换及 BGR\1 响应。
[X]	同一 revision: AKPlugin.swift, 124-168 行	ScreenCaptureKit 与旧版 CGWindow 图像采集路径。
[X]	同一 revision: PlaySettings.swift, 13-23、76-78 行	外部进程读取宿主 plist 及 MaaTools 配置。
固 定	revision	参 考: https://github.com/hguandl/PlayTools/tree/7aa19776dbb72b0ae7f0989568f69c7c9c2d6543 。外部源码在当前 PlayCover checkout 中没有以 Carthage/Build 形式存在。

B 验证状态与后续测试矩阵

本报告已完成宿主侧和固定 revision 外部源码的静态核对。由于当前工作树没有构建好的 PlayTools framework，以下运行时项目尚未宣称通过：

项目	需要的验证
协议握手与端序	构建目标应用，连接 MaaTools，验证 MAA\0/OKAY、2 字节大端序和各响应字段。
像素格式 权限行为	分别请求 SCR_N 与 BGR\1，检查尺寸、长度、通道顺序和裁剪区域。在允许和拒绝 Screen Recording 的 TCC 状态下比较日志、空数据和用户反馈。
网络边界	用 socket 工具确认 listener 的本地/非本地绑定范围，测试端口冲突和重复连接。
生命周期 控制命令	测试窗口迟迟不出现、窗口关闭、目标进程退出和 listener 取消。在受控环境中验证 TOUCH 和 TERM 的授权边界，避免与截图权限混用。